Optimization Methods: Computing the ERM Predictor $h_{\mathcal{T}}$

Lecture Notes

Giorgio Gambosi
Master's Degree in Computer Science
University of Rome "Tor Vergata"

A.Y. 2026–2027

Contents

1 From Statistical Guarantees to Numerical Optimization **1**

2 Gradient Descent **2**

2.1 The Update Rule and Its Four Steps . . . 2
2.2 Batch Gradient Descent . . . 2
2.3 Stochastic Gradient Descent . . . 3
2.4 Mini-Batch Gradient Descent . . . 4

3 Momentum and Acceleration **4**

3.1 Momentum . . . 4
3.2 Nesterov Accelerated Gradient . . . 5

4 Adaptive Learning Rate Methods **5**

4.1 Per-Parameter Rates and Exponential Moving Averages . . . 5
4.2 Adagrad, RMSprop and Adadelta . . . 6
4.3 Adam . . . 7
4.4 Comparing Adaptive Methods . . . 7
4.5 Practical Safeguards: Gradient Clipping and Warmup . . . 8
4.6 Weight Decay and AdamW . . . 8

5 Second-Order Methods **9**

5.1 Newton's Method . . . 9
5.2 Computational Cost and Quasi-Newton Methods . . . 10
5.3 Convergence Rates Compared . . . 10

6 Why Second-Order Methods Do Not Dominate Deep Learning **10**

1 From Statistical Guarantees to Numerical Optimization

The Fundamental Theorem guarantees that ERM finds a near-optimal predictor whenever the VC dimension of $\mathcal{H}$ is finite, but says nothing about whether that predictor can be *computed* efficiently. In the terms used when designing losses, the loss fixes *what* is estimated; this chapter deals with the second level, *how* the minimizer of the empirical risk is actually found. In most practical settings $\mathcal{H}$ is parameterized by $\boldsymbol{\theta} \in \mathbb{R}^d$, and minimizing $\overline{\mathcal{R}}_{\mathcal{T}}(h_{\boldsymbol{\theta}})$ becomes an unconstrained optimization problem over $\mathbb{R}^d$. The function actually minimized is usually not the empirical risk under the loss of interest, but under a convex surrogate, as in classification, often with a penalty term, which corresponds to a prior on the parameters (MAP estimation, in the notes on the inference of probabilistic

models). Closed-form solutions exist only in special cases, the main one being linear regression under the quadratic loss, whose normal equations were derived in those notes; for logistic regression, as also seen there, no closed form exists, and in general iterative numerical methods are needed.

A necessary condition for a minimizer is stationarity, $\nabla\overline{\mathcal{R}}_{\mathcal{T}}(h_{\boldsymbol{\theta}}) = \mathbf{0}$: no direction in parameter space decreases the objective to first order. When the objective is convex in $\boldsymbol{\theta}$, any stationary point is a global minimum; otherwise it may be a local minimum, a local maximum or a saddle point. For linear models, the quadratic, logistic and hinge losses and the log score all give convex objectives: this is the computational reason why convex surrogates were required in classification. For neural networks the objective is never convex. This distinction becomes central when discussing why deep networks are hard to optimize (Section 6).

2 Gradient Descent

2.1 The Update Rule and Its Four Steps

Gradient descent moves iteratively against the gradient of the objective $J(\boldsymbol{\theta})$,

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta\,\nabla J\big|_{\boldsymbol{\theta}^{(k)}}, \qquad \text{componentwise: } \theta_j^{(k+1)} = \theta_j^{(k)} - \eta\frac{\partial J(\boldsymbol{\theta})}{\partial\theta_j}\Big|_{\boldsymbol{\theta}^{(k)}}. \tag{1}$$

The learning rate η trades speed against stability: if it is too large the iterates overshoot, oscillate or diverge; if it is too small progress is slow.

It is convenient to split each update into four steps:

$$\begin{aligned} g_{j,k} &= \frac{\partial J(\boldsymbol{\theta})}{\partial\theta_j}\Big|_{\boldsymbol{\theta}^{(k)}} && \text{(gradient signal)}\\ \eta_{j,k} &= \eta && \text{(learning rate)}\\ \Delta_{j,k} &= -\eta_{j,k}\,g_{j,k} && \text{(increment)}\\ \theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k} && \text{(update)} \end{aligned}$$

Plain gradient descent uses a single constant η, shared by all parameters and iterations. The methods of this chapter can be classified by the step they change: batch, stochastic and mini-batch gradient descent differ only in how the gradient signal $g_{j,k}$ is estimated, that is, in how many training examples are used per update; momentum changes the increment, combining the current gradient with past ones; adaptive methods change the learning rate, which becomes specific to each parameter and iteration (Adam changes both rate and increment); second-order methods replace the scheme altogether, using curvature information.

2.2 Batch Gradient Descent

In batch gradient descent the objective is the full empirical risk,

$$J(\boldsymbol{\theta}) = \overline{\mathcal{R}}_{\mathcal{T}}(h_{\boldsymbol{\theta}}) = \frac{1}{|\mathcal{T}|}\sum_{(\mathbf{x},t)\in\mathcal{T}} L(h_{\boldsymbol{\theta}}(\mathbf{x}),t), \qquad g_{j,k} = \frac{1}{|\mathcal{T}|}\sum_{(\mathbf{x},t)\in\mathcal{T}}\frac{\partial L(h_{\boldsymbol{\theta}}(\mathbf{x}),t)}{\partial\theta_j}\Big|_{\boldsymbol{\theta}^{(k)}} \tag{2}$$

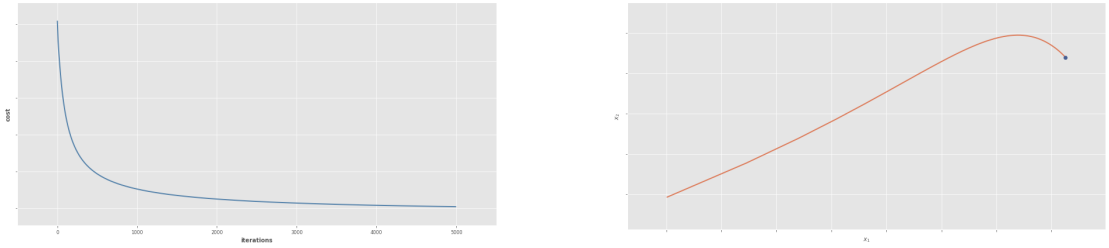
Averaging over all examples gives an exact gradient of the empirical risk, with no sampling noise.

For linear regression, $h(\mathbf{x}) = \sum_s \theta_s x_s + \theta_0$, with the quadratic loss written as $L = \frac{1}{2}(h(\mathbf{x}) - t)^2$, the residual $r = \sum_s \theta_s^{(k)} x_s + \theta_0^{(k)} - t$ gives $\dfrac{\partial L}{\partial\theta_j} = r\,x_j$ and $\dfrac{\partial L}{\partial\theta_0} = r$, so that

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \frac{\eta}{|\mathcal{T}|}\sum_{(\mathbf{x},t)\in\mathcal{T}} r\,x_j, \qquad \theta_0^{(k+1)} = \theta_0^{(k)} - \frac{\eta}{|\mathcal{T}|}\sum_{(\mathbf{x},t)\in\mathcal{T}} r. \tag{3}$$

Each example pushes θ_j in the direction that reduces its residual. If the prediction is too high ($r > 0$) and $x_j > 0$, decreasing θ_j lowers the prediction, and the update does so. If r and x_j have opposite signs, the update increases θ_j, which again moves the prediction toward the target. This residual-times-feature form recurs in all the variants below; only the set of examples over which it is averaged changes.

Batch gradient descent follows a smooth trajectory and, for a suitable η, decreases the cost at every step (Figure 1). It converges to the global minimum when J is convex, and to a stationary point, typically a local minimum, otherwise. Its drawback is that each update requires a full pass over $\mathcal{T}$, which is too expensive for large datasets and does not fit online learning, where data arrives as a stream.



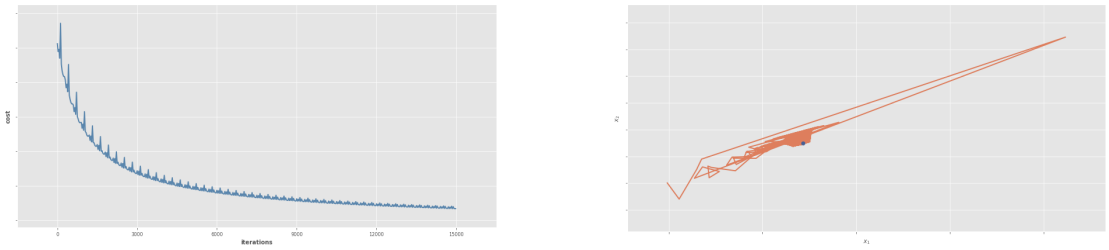
(a) Cost vs. iterations (b) Trajectory

Figure 1: Batch gradient descent: the cost decreases at every step and the trajectory is direct, since the full-dataset gradient carries no noise.

2.3 Stochastic Gradient Descent

Stochastic gradient descent (SGD) uses a single example $(\mathbf{x}, t)$, drawn at random, per update: $J(\boldsymbol{\theta}) = L(h_{\boldsymbol{\theta}}(\mathbf{x}), t)$ and $g_{j,k} = \frac{\partial}{\partial \theta_j} L(h_{\boldsymbol{\theta}}(\mathbf{x}), t)$. This is an unbiased estimate of the batch gradient, since its expectation over the choice of the example is the batch gradient, for the same reason that the empirical risk is an unbiased estimate of the risk. Its variance, however, is high, because it reflects a single data point. For linear regression the update has the same form as before, with the average over $\mathcal{T}$ replaced by a single term: $\theta_j^{(k+1)} = \theta_j^{(k)} - \eta\, r\, x_j$.

The price of this estimate is a noisy cost curve and an erratic trajectory (Figure 2). On the other hand, each update costs $|\mathcal{T}|$ times less than a batch update, updates are much more frequent, and streaming data is handled naturally. The noise can also help: on landscapes with shallow local minima or saddle points, it can push the iterate out of regions where batch gradient descent, which follows the exact gradient, would stop.



(a) Cost vs. iterations (b) Trajectory

Figure 2: Stochastic gradient descent: a noisy cost curve and an erratic trajectory, the price of a single-example gradient estimate.

With a constant learning rate, SGD does not converge to a minimum: the iterates keep fluctuating in a region around it, whose size grows with η. Convergence requires a decreasing schedule

η_k, and the classical sufficient conditions are

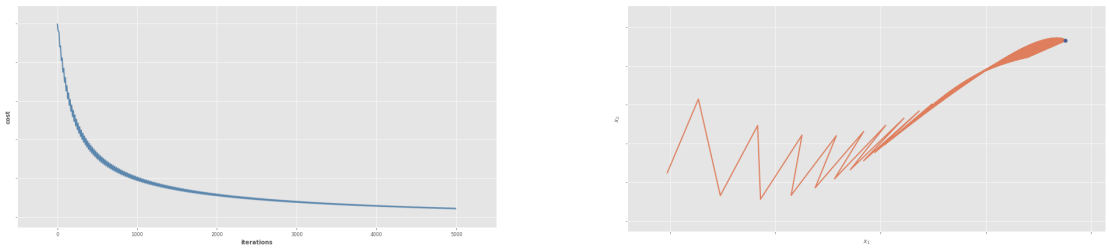
$$\sum_k \eta_k = \infty, \qquad \sum_k \eta_k^2 < \infty. \tag{4}$$

2.4 Mini-Batch Gradient Descent

Mini-batch gradient descent averages over a random subset $B \subset \mathcal{T}$ of size m,

$$J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{(\mathbf{x},t)\in B} L(h_{\boldsymbol{\theta}}(\mathbf{x}), t). \tag{5}$$

This reduces the variance of the SGD gradient by a factor close to m, while each update remains much cheaper than a full pass. The trajectory is correspondingly smoother than that of SGD (Figure 3). The residual-times-feature update carries over, now averaged over B. Setting $m = 1$ gives SGD, and $m = |\mathcal{T}|$ gives batch gradient descent. Typical values of m range from 32 to 256. In this range the m per-example gradients are computed in parallel on a GPU, so that a mini-batch update takes not much longer than a single-example one; as a result, mini-batch gradient descent is often the fastest option in practice, and not just a compromise between the other two. In deep learning, "SGD" normally refers to this mini-batch variant.



(a) Cost vs. iterations

(b) Trajectory

Figure 3: Mini-batch gradient descent: smoother than SGD, cheaper per step than batch gradient descent, and parallelizable on GPUs.

3 Momentum and Acceleration

3.1 Momentum

The three variants above share a limitation: each update depends only on the current gradient, with no memory of previous steps. In a narrow ravine, where the gradient points mostly *across* the valley and only weakly *along* it, this produces a zigzag: each step overshoots across the ravine, and progress along its floor is slow (Figure 4, left).

Momentum addresses this through a physical analogy: $\boldsymbol{\theta}$ is the position of a unit-mass body subject to the force $F = -\eta \nabla J(\boldsymbol{\theta})$. Instead of restarting from zero velocity at each step, as plain gradient descent implicitly does, the body moves with an accumulated velocity:

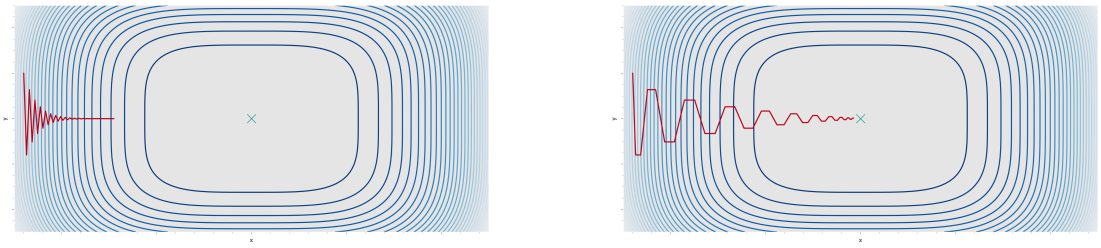
$$v_j^{(k+1)} = \gamma v_j^{(k)} - \eta\, g_{j,k}, \qquad \theta_j^{(k+1)} = \theta_j^{(k)} + v_j^{(k+1)}. \tag{6}$$

The damping factor $\gamma \in [0, 1)$ (typically 0.9 or 0.99) prevents the velocity from growing without bound on a long slope; with $\gamma = 1$, v would add up all past gradients with no decay.

Unrolling the recursion from $v_j^{(0)} = 0$ gives

$$v_j^{(k+1)} = -\eta \sum_{r=0}^{k} \gamma^{k-r} g_{j,r} : \tag{7}$$

each increment is a geometrically weighted sum of past gradients, with more weight on recent ones. On a consistent slope the gradients agree in sign, the velocity builds up and the descent accelerates. Across a ravine the gradients alternate in sign and their contributions largely cancel, which damps the oscillation, while the component along the floor of the ravine accumulates (Figure 4, right). For linear regression, the increment is a discounted sum of past residual-times-feature terms.



(a) Without momentum

(b) With momentum

Figure 4: Momentum damps the oscillation across the ravine and accumulates speed along its floor.

3.2 Nesterov Accelerated Gradient

Nesterov accelerated gradient (NAG) changes the point where the gradient is evaluated. Since the momentum term alone would bring the iterate to the look-ahead position $\tilde{\boldsymbol{\theta}}^{(k)} = \boldsymbol{\theta}^{(k)} + \gamma \mathbf{v}^{(k)}$, the gradient is computed there:

$$v_j^{(k+1)} = \gamma v_j^{(k)} - \eta\, \tilde{g}_{j,k}, \qquad \tilde{g}_{j,k} = \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j}\Big|_{\tilde{\boldsymbol{\theta}}^{(k)}}, \qquad \theta_j^{(k+1)} = \theta_j^{(k)} + v_j^{(k+1)}. \tag{8}$$

If the momentum is about to carry the iterate past the minimum, the gradient at the look-ahead point already reflects the rising slope beyond it, and the correction is applied before the overshoot rather than after. Figure 5 compares the three steps taken from the same point by gradient descent, momentum and NAG.

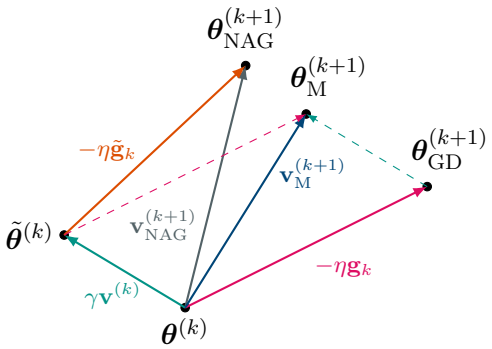


Figure 5: One step from $\boldsymbol{\theta}^{(k)}$ with gradient descent (GD), momentum (M) and Nesterov accelerated gradient (NAG). Momentum adds $\gamma\mathbf{v}^{(k)}$ to the gradient step $-\eta\mathbf{g}_k$ computed at $\boldsymbol{\theta}^{(k)}$; NAG first moves to the look-ahead point $\tilde{\boldsymbol{\theta}}^{(k)}$ and computes the gradient step $-\eta\tilde{\mathbf{g}}_k$ there.

4 Adaptive Learning Rate Methods

4.1 Per-Parameter Rates and Exponential Moving Averages

A single global η, however it is scheduled during training (step decay, exponential decay, $1/k$ decay), applies equally to all parameters, while the gradient statistics of different parameters can be very

different: a weight attached to a frequent feature receives frequent, large gradients, one attached to a rare feature receives sparse, small ones. A rate suited to the first is too small for the second, and a rate suited to the second may destabilize the first. Adaptive methods give each parameter its own effective rate, computed from its own gradient history.

All adaptive methods are built on the **exponential moving average** (EMA). For a sequence $f^{(0)}, f^{(1)}, \dots$ and a decay $\gamma \in (0,1)$,

$$\tilde{F}_k = \gamma \tilde{F}_{k-1} + (1-\gamma) f^{(k)} = (1-\gamma) \sum_{i=0}^{k} \gamma^{k-i} f^{(i)}, \qquad \tilde{F}_{-1} = 0. \tag{9}$$

The value $f^{(i)}$ enters with weight $(1-\gamma)\gamma^{k-i}$, which decays geometrically with its age: for $\gamma = 0.9$, a value fifty steps old has weight $0.1 \cdot 0.9^{50} \approx 5 \cdot 10^{-4}$, that is, 0.005 times the weight of the most recent one. With $f = g^2$ the EMA tracks the scale of the gradient, with $f = g$ the gradient itself, playing the role of momentum.

4.2 Adagrad, RMSprop and Adadelta

Adagrad uses a parameter-specific rate that decreases as θ_j accumulates gradient signal: in order to deal with poositive or negative values, the squares of previous gradients are summed and a small constant ε is also added to avoid a zero result. The running sum is then $G_{j,k} = \sum_{i=0}^{k} g_{j,i}^2$ and the learning rate is inversely proportional to the square root

$$\eta_{j,k} = \frac{\eta}{\sqrt{G_{j,k} + \varepsilon}}. \tag{10}$$

Parameters that have received large gradients take smaller steps, while those that have received little signal keep larger ones. The constant ε (typically 10^{-8}) avoids divisions by zero, but also bounds the rate of parameters whose gradients remain very small. The limitation of Adagrad is that $G_{j,k}$ can only grow: the rate decreases monotonically and, in long training runs, becomes too small to make progress well before convergence.

RMSprop replaces the running sum with an EMA of squared gradients,

$$\tilde{G}_{j,k} = \gamma \tilde{G}_{j,k-1} + (1-\gamma) g_{j,k}^2, \qquad \eta_{j,k} = \frac{\eta}{\sqrt{\tilde{G}_{j,k} + \varepsilon}}. \tag{11}$$

Since $\tilde{G}_{j,k}$ only reflects recent gradients, it does not grow without bound: if gradients become smaller, $\tilde{G}_{j,k}$ decreases and the effective rate increases again. Typical values are $\gamma \simeq 0.9$, $\eta \simeq 10^{-3}$, $\varepsilon \simeq 10^{-8}$; the global rate η still has to be tuned.

Adadelta removes η through a dimensional argument: the increment $\Delta_{j,k}$ should have the units of θ_j, while in RMSprop it has the units of η, since $g_{j,k}/\sqrt{\tilde{G}_{j,k}}$ is dimensionless. Adadelta replaces η with the root of an EMA of past squared increments, $\overline{G}_{j,k} = \gamma \overline{G}_{j,k-1} + (1-\gamma)\Delta_{j,k}^2$:

$$\eta_{j,k} = \frac{\sqrt{\overline{G}_{j,k-1} + \varepsilon}}{\sqrt{\tilde{G}_{j,k} + \varepsilon}}. \tag{12}$$

The numerator uses $\overline{G}_{j,k-1}$, since $\overline{G}_{j,k}$ depends on the increment $\Delta_{j,k}$ that is being computed. The rate is the ratio between the recent scale of the increments and the recent scale of the gradients, and needs no global learning rate (typically $\gamma \simeq 0.95$). Like Adagrad and RMSprop, Adadelta adapts only the magnitude of the steps, not their direction, which is still that of the current gradient.

4.3 Adam

Adam (Adaptive Moment Estimation) combines the second-moment scaling of RMSprop with a first-moment (momentum) term, both maintained as EMAs:

$$\tilde{H}_{j,k} = \beta_1 \tilde{H}_{j,k-1} + (1-\beta_1) g_{j,k}, \qquad \tilde{G}_{j,k} = \beta_2 \tilde{G}_{j,k-1} + (1-\beta_2) g_{j,k}^2, \tag{13}$$

with typical values $\beta_1 \simeq 0.9$ and $\beta_2 \simeq 0.999$ (the names used in the original paper and in software libraries). Both averages start from zero, and when β_1 and β_2 are close to 1 the early estimates are biased toward zero. At $k=0$, for instance, $\tilde{G}_{j,0} = (1-\beta_2) g_{j,0}^2$ is a thousandth of $g_{j,0}^2$. After $k+1$ terms the weights of an EMA with decay β sum to $1-\beta^{k+1}$, and the bias is removed by dividing by this sum:

$$\hat{H}_{j,k} = \frac{\tilde{H}_{j,k}}{1-\beta_1^{k+1}}, \qquad \hat{G}_{j,k} = \frac{\tilde{G}_{j,k}}{1-\beta_2^{k+1}}. \tag{14}$$

At $k=0$ the correction divides by $1-\beta_2 = 0.001$ and restores the scale of $g_{j,0}^2$. It fades out at different speeds for the two averages. For $\beta_1 = 0.9$ it is negligible after a few dozen steps; for $\beta_2 = 0.999$ it is still a factor $1/(1-0.999^{1001}) \approx 1.6$ after a thousand steps, and becomes negligible only after several thousand. The update is

$$\eta_{j,k} = \frac{\eta}{\sqrt{\hat{G}_{j,k} + \varepsilon}}, \qquad \Delta_{j,k} = -\eta_{j,k}\, \hat{H}_{j,k}. \tag{15}$$

The direction comes from the smoothed gradient $\hat{H}_{j,k}$, which reduces the effect of noise. The scale comes from the per-parameter estimate $\hat{G}_{j,k}$, which compensates for heterogeneous gradient magnitudes. When the gradient keeps a consistent sign, $\hat{H}_{j,k}/\sqrt{\hat{G}_{j,k}}$ is of order 1, so each parameter moves by roughly η per step, whatever the scale of its gradient. This robustness makes Adam, with defaults $\eta \simeq 10^{-3}$, $\beta_1 \simeq 0.9$, $\beta_2 \simeq 0.999$, the standard first choice in deep learning.

A worked comparison: gradient descent versus Adam

Consider a single parameter and $J(\theta) = (\theta-3)^2$, so that $g_k = 2(\theta^{(k)} - 3)$, starting from $\theta^{(0)} = 0$.

Gradient descent with $\eta = 0.1$: $g_0 = -6$, so $\theta^{(1)} = 0 - 0.1 \cdot (-6) = 0.6$; then $g_1 = -4.8$ and $\theta^{(2)} = 1.08$. At each step the distance from the minimum is multiplied by $1-2\eta = 0.8$.

Adam with $\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$: at $k=0$, $\tilde{H}_0 = 0.1 \cdot (-6) = -0.6$ and $\tilde{G}_0 = 0.001 \cdot 36 = 0.036$; after bias correction $\hat{H}_0 = -6$ and $\hat{G}_0 = 36$, the raw gradient and its square. The increment is $\Delta_0 = -0.1 \cdot (-6)/\sqrt{36} = 0.1$, so $\theta^{(1)} = 0.1$. At the next step $g_1 = -5.8$, $\hat{H}_1 \approx -5.89$, $\hat{G}_1 \approx 34.8$, and $\theta^{(2)} \approx 0.2$.

Adam moves by roughly η per step, independently of the size of the gradient. Here gradient descent is faster, because the gradient is large ($|g| \approx 6$). With gradients a hundred times smaller, gradient descent would move a hundred times more slowly, while the steps of Adam would be unchanged.

Convenience does not imply better final models. Several empirical studies have found that plain SGD with momentum, although harder to tune, sometimes yields models that generalize better than those trained with Adam on the same architecture, particularly in computer vision. In practice, the choice between the two is made empirically, for each architecture and task.

4.4 Comparing Adaptive Methods

The adaptive methods form a cumulative progression: Adagrad introduces per-parameter rates; RMSprop prevents their decay to zero by replacing the sum with an EMA; Adadelta removes the global rate through a second EMA; Adam adds a momentum EMA for the direction, and bias correction. All of them fit the scheme $\Delta_{j,k} = -\eta_{j,k}\, d_{j,k}$, with the rate and direction of Table 1.

Method	Rate $\eta_{j,k}$	Direction $d_{j,k}$	Typical values
GD / SGD	η	$g_{j,k}$	—
Adagrad	$\dfrac{\eta}{\sqrt{G_{j,k}+\varepsilon}}$	$g_{j,k}$	$\eta \simeq 10^{-2}$
RMSprop	$\dfrac{\eta}{\sqrt{\tilde{G}_{j,k}+\varepsilon}}$	$g_{j,k}$	$\eta \simeq 10^{-3}, \gamma \simeq 0.9$
Adadelta	$\dfrac{\sqrt{\overline{G}_{j,k-1}+\varepsilon}}{\sqrt{\tilde{G}_{j,k}+\varepsilon}}$	$g_{j,k}$	$\gamma \simeq 0.95$
Adam	$\dfrac{\eta}{\sqrt{\hat{G}_{j,k}+\varepsilon}}$	$\hat{H}_{j,k}$	$\eta \simeq 10^{-3}, \beta_1 \simeq 0.9, \beta_2 \simeq 0.999$

Table 1: Adaptive methods as choices of rate and direction. G: running sum of g^2; $\tilde{G}$: EMA of g^2; $\overline{G}$: EMA of Δ^2; $\hat{G}$, $\hat{H}$: bias-corrected EMAs of g^2 and g.

4.5 Practical Safeguards: Gradient Clipping and Warmup

Deep networks, especially in early training or with momentum accumulating velocity, occasionally produce very large gradients (an unusual example, an unstable layer) that can move $\boldsymbol{\theta}$ far in a single step. Gradient clipping rescales the gradient whenever its norm exceeds a threshold c,

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\Big(1, \frac{c}{\|\mathbf{g}\|}\Big), \tag{16}$$

leaving ordinary gradients unchanged. As observed when designing losses, it bounds the influence of a single example on the update, as a Lipschitz loss would. It is not an alternative to the methods above, but a safeguard used together with any of them.

A second safeguard concerns adaptive methods in the first steps. There, $\hat{G}_{j,k}$ is estimated from a handful of gradients: after bias correction it is unbiased, but its variance is high. Since each parameter moves by roughly η regardless of how informative its gradient is, the first updates can be large and erratic. A learning-rate warmup starts from a small η and increases it, typically linearly, over the first hundreds or thousands of steps, until the second-moment estimates are reliable.

4.6 Weight Decay and AdamW

Adaptive rates also interact with regularization. An L_2 term is normally added to the objective, $J(\boldsymbol{\theta}) = \overline{\mathcal{R}}_{\mathcal{T}}(h_{\boldsymbol{\theta}}) + \lambda\|\boldsymbol{\theta}\|^2$, so that $g_{j,k}$ includes a contribution $2\lambda\theta_j$. For plain gradient descent the update then becomes

$$\theta_j^{(k+1)} \;=\; \theta_j^{(k)} - \eta\Big(\frac{\partial \overline{\mathcal{R}}_{\mathcal{T}}}{\partial \theta_j} + 2\lambda\theta_j^{(k)}\Big) \;=\; (1-2\eta\lambda)\,\theta_j^{(k)} - \eta\,\frac{\partial \overline{\mathcal{R}}_{\mathcal{T}}}{\partial \theta_j}. \tag{17}$$

Before the step dictated by the data through the empirical risk and its gradient, each parameter is multiplied by $1-2\eta\lambda < 1$, that is, it loses a fixed fraction $2\eta\lambda$ of its current value. Without the data term, k such steps would give $\theta_j^{(k)} = (1-2\eta\lambda)^k\,\theta_j^{(0)}$, which goes to zero geometrically: the parameters decay toward zero, larger ones losing more in absolute terms. With the data term, a parameter stays away from zero only as long as the gradient of the empirical risk keeps pushing it there, and its final value balances the two effects. This is called weight decay: for plain gradient descent, adding the L_2 term and shrinking each parameter by a fixed fraction at every step are the same thing.

They no longer coincide with an adaptive method. Adam divides each component of the gradient by $\sqrt{\hat{G}_{j,k} + \varepsilon}$, the L_2 contribution included, so that the shrinking actually applied to θ_j is

$$\frac{2\eta\lambda}{\sqrt{\hat{G}_{j,k} + \varepsilon}}\,\theta_j \tag{18}$$

instead of $2\eta\lambda\,\theta_j$: each parameter is regularized with its own effective coefficient, smaller for parameters with a large gradient history, larger for those with a small one. A uniform penalty becomes a non-uniform one, chosen by the gradients rather than by λ. AdamW decouples the two operations: the Adam update is computed from the gradient of the loss alone, and each parameter is then shrunk by the same fraction, $\theta_j \leftarrow (1 - 2\eta\lambda)\,\theta_j$, as in plain gradient descent (libraries usually absorb the factor 2 into their weight-decay coefficient). The update is then no longer a step on a fixed objective. AdamW is now the standard choice for training transformers and other large models.

5 Second-Order Methods

5.1 Newton's Method

Gradient descent uses only the local slope of J, ignoring how the slope changes. A second-order expansion around $\boldsymbol{\theta}^{(k)}$,

$$J(\boldsymbol{\theta}^{(k)} + \delta) \approx J(\boldsymbol{\theta}^{(k)}) + \nabla J(\boldsymbol{\theta}^{(k)})^\top \delta + \tfrac{1}{2}\delta^\top H(J)(\boldsymbol{\theta}^{(k)})\delta, \tag{19}$$

where $H(J)_{ij} = \dfrac{\partial^2 J}{\partial\theta_i \partial\theta_j}$ is the Hessian, includes this information. Minimizing the quadratic model gives the Newton step

$$\delta^* = -H(J)(\boldsymbol{\theta}^{(k)})^{-1}\nabla J(\boldsymbol{\theta}^{(k)}), \qquad \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \delta^*, \tag{20}$$

which in one dimension reads $\theta^{(k+1)} = \theta^{(k)} - \dfrac{J'(\theta^{(k)})}{J''(\theta^{(k)})}$. Gradient descent follows the tangent plane, with a step length fixed by η; Newton's method fits a paraboloid and moves to its minimum (Figure 6). Since the curvature in every direction is taken into account, the Newton step is invariant to affine transformations of the parameters, while gradient descent is sensitive to their scaling.

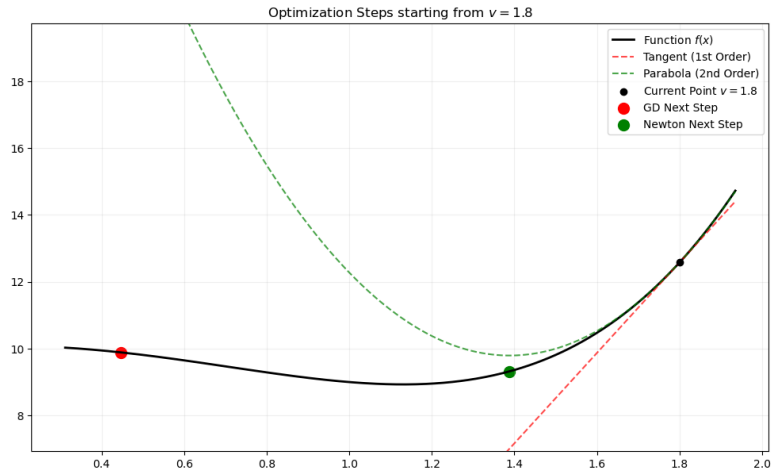


Figure 6: Gradient descent steps along the tangent plane; Newton's method steps to the minimum of the fitted quadratic model.

When J is quadratic, one Newton step reaches the minimum. More generally, near a minimum where the Hessian is positive definite and sufficiently smooth, Newton's method converges quadratically: the error is roughly squared at each step ($10^{-2} \to 10^{-4} \to 10^{-8}$). Gradient descent, on a strongly convex function, converges only linearly, the error being multiplied by a constant factor at each step. Far from the minimum the quadratic model may be inaccurate and the full step may overshoot; a damped step $\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \eta\delta^*$, with $\eta < 1$, controls this.

5.2 Computational Cost and Quasi-Newton Methods

For d parameters, storing H requires $O(d^2)$ memory, and solving $H\delta = -\nabla J$ requires $O(d^3)$ time: feasible for d in the thousands, not for modern networks. Quasi-Newton methods build an approximation of H^{-1} from successive gradients, without computing the Hessian. BFGS converges superlinearly but still stores a $d \times d$ matrix; L-BFGS keeps only the last m pairs of parameter and gradient differences ($m \sim 10$–20), with $O(dm)$ memory and cost per step, and in theory converges only linearly, although it is very effective in practice on deterministic problems. L-BFGS, however, relies on line searches, which assume that the objective is the same at every evaluation. With mini-batches this does not hold: successive gradients are computed on different subsets of the data, and the apparent objective fluctuates independently of the step. This is a first reason why second-order methods are rarely used in deep learning; the others are discussed in Section 6.

5.3 Convergence Rates Compared

Table 2 summarizes rates and costs. Rates are stated under the assumptions for which they are proved: L-smooth convex or strongly convex objectives for first-order methods, a neighborhood of a minimum with positive definite Hessian for Newton's method. In the stochastic setting, the rates of adaptive methods are sublinear, and Adam may even fail to converge on some convex problems (a fact that motivated variants such as AMSGrad).

Method	Rate	Cost/step	Memory
Gradient descent	$O(1/k)$ convex; linear strongly convex	$O(d\,\lvert\mathcal{T}\rvert)$	$O(d)$
SGD, η_k decreasing	sublinear, e.g. $O(1/\sqrt{k})$ convex	$O(d)$	$O(d)$
Nesterov	$O(1/k^2)$ convex; linear strongly convex, better constant	as GD	$O(d)$
Adagrad, RMSprop, Adam	sublinear (stochastic); no general guarantee for Adam	$O(d)$	$O(d)$
L-BFGS	linear (theory), fast in practice	$O(dm)$	$O(dm)$
BFGS	superlinear	$O(d^2)$	$O(d^2)$
Newton	quadratic near a minimum	$O(d^3)$	$O(d^2)$

Table 2: Convergence rates and costs; d is the number of parameters, m the memory of L-BFGS.

Faster convergence per iteration is paid for with more expensive iterations. Which end of the table is preferable depends on whether d is small enough, and gradients accurate enough, for the cost to be worth it.

6 Why Second-Order Methods Do Not Dominate Deep Learning

Although second-order methods take a more direct route to a minimum, deep networks are trained almost exclusively with first-order methods, for several reasons.

The first is **scale**. The Hessian of a network with d parameters has d^2 entries, far too many to store or invert for current models; even L-BFGS, which avoids storing it, needs several gradi-

ent evaluations per step and parallelizes poorly on GPUs, while Adam needs a single gradient and memory linear in d.

The second is **mini-batch noise**. A Newton step rescales the gradient by the curvature in each direction: it takes long steps where the loss is flat and short steps where it is steep. Along flat directions, however, the true gradient is small, and the gradient computed on a mini-batch is dominated by noise; dividing it by a small curvature turns this noise into a long, essentially random step. The curvature is estimated on the same mini-batch, so it is noisy as well, and an underestimated curvature makes the step even longer. Adam, instead, averages the gradients over many steps, so that the noise partly cancels, and scales each component by the typical size of its recent gradients, so that noisy components receive smaller steps.



(a) Newton's method (b) Gradient descent

Figure 7: The first 40 iterates of Newton's method and of gradient descent with noisy mini-batch gradients, from the same starting point (square), on a loss that is flat horizontally and steep vertically; the star is the minimum. Without noise, Newton's method would reach the minimum in one step; with noise, each step divides the noise by the small curvature of the flat direction, and the iterates keep jumping back and forth along it, far from the minimum. Gradient descent moves more slowly, but steadily, and remains close to the minimum once it gets there.

The third is the presence of **saddle points**, points where the gradient vanishes and the loss curves upward along some directions and downward along others, like a mountain pass. In high dimensions they are much more frequent than local minima, since a minimum requires the loss to curve upward along every direction. Newton's method approximates the loss with a quadratic and jumps to the point where its gradient vanishes, without checking whether it is a minimum: near a saddle, that point is the saddle itself, so along the directions where the loss curves downward the step moves uphill, toward the saddle. Gradient descent only follows the slope downhill: near a saddle the slope is small and progress slows down, but along the directions of downward curvature it moves away from the saddle, and the noise of mini-batches helps it escape.

The last reason concerns **generalization**. A deep network has many minima of the training loss with similar values, and what matters is reaching one that also performs well on new data. A common, although debated, view relates this to the shape of the minimum. The loss computed on new data differs from the training loss, roughly as if the training loss were slightly shifted (Figure 9a). At a flat minimum, where the loss changes slowly around the minimizer, such a shift barely changes its value; at a sharp minimum, in a narrow valley, the same shift raises it considerably. The noise of SGD makes the iterates bounce around: in a narrow valley they are easily kicked out, while in a wide one they remain, so SGD tends to end in flat minima. Second-order methods, instead, go straight to the nearest minimum, whatever its shape, and in a narrow valley the large curvature divides the noise of their steps, so that they remain there (Figure 9b). This view is only a heuristic: how flat a minimum looks depends on how the network is parameterized, since rescaling the parameters can make the same function look flatter or sharper.

In practice, generalization is also controlled by early stopping. During training, the training error keeps decreasing; the error on a validation set, as in the foundations notes, decreases at first and then starts to increase, when the parameters begin to fit the noise of the training set. Early stopping ends training at that point and keeps the parameters with the smallest validation error. It

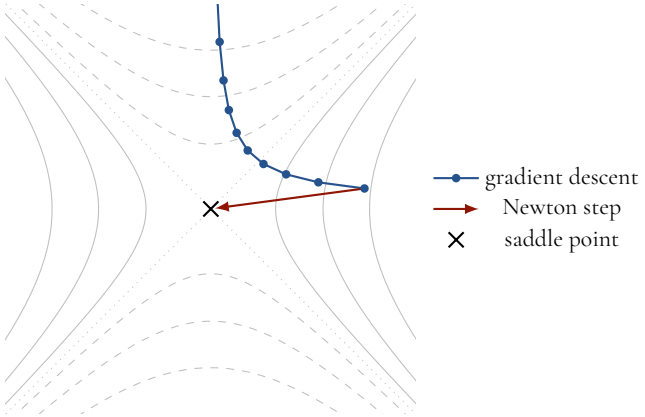
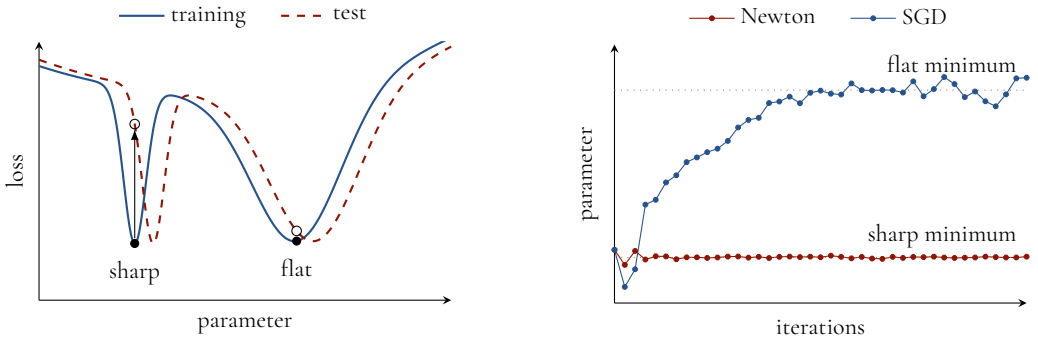


Figure 8: Level curves of a loss with a saddle point at the origin: the loss increases moving horizontally away from it (solid curves) and decreases moving vertically (dashed curves). From the same starting point, Newton's method jumps to the saddle in one step, while gradient descent slows down near it and then moves away along the vertical direction, where the loss keeps decreasing.

acts as an implicit regularization: stopping early keeps the parameters close to their initialization, much as an L_2 penalty keeps them close to zero.



(a) Sharp and flat minima

(b) Newton's method and SGD

Figure 9: (a) Two minima of the training loss with nearly the same value. The test loss, computed on different data, is shown as a slightly shifted copy of it: at the sharp minimum the shift raises the loss considerably (circle), at the flat one only slightly. (b) The parameter along the iterations, from the same starting point inside the sharp valley and with the same level of gradient noise. Newton's method settles at the sharp minimum, the nearest one; SGD keeps bouncing in the narrow valley, is soon kicked out, and settles at the flat minimum.

In deep learning, the trade-off between speed, memory and robustness to noise thus favors first-order methods: Adam is a convenient default, and SGD with momentum a competitive alternative.

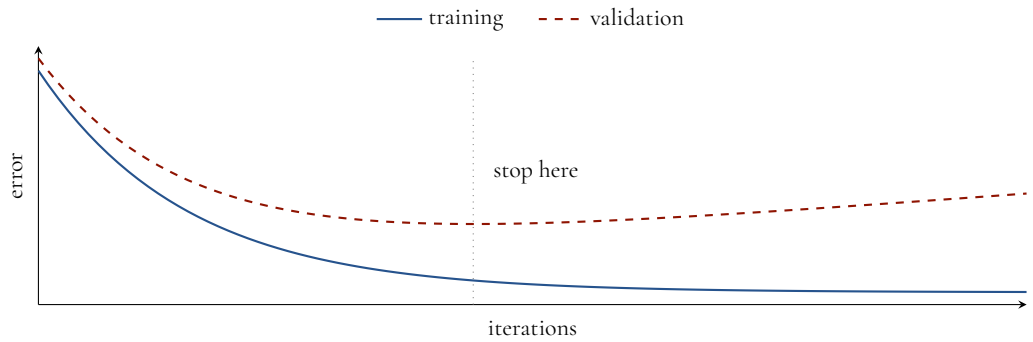


Figure 10: The training error keeps decreasing, while the validation error starts to increase once the parameters begin to fit the noise of the training set; early stopping keeps the parameters at the minimum of the validation error.